# Tutorial 2- 2026-2027

Exercise 1:
Consider the following `ProductInventory` table from a company's relational database:

| product_code | product_name | supplier_cost | clearance_item | warehouse_zone |
|---|---|---|---|---|
| PROD-881 | Ergonomic Chair | 145.50 | False | Zone B |
| PROD-902 | Mechanical Keyboard | 65.00 | True | Zone A |
| PROD-445 | UltraWide Monitor | 320.00 | False | Zone C |

- **a.** How many records and fields are in this table?
- **b.** Identify the most appropriate data type of each field

Exercise 2:

- **a.** For each of the following strategic data requests in a **Hospital Management System**, identify the necessary data fields that must be collected in the database:
  1. Monthly Total Overtime Pay for nurses.
  2. Medical suppliers who delivered surgical gowns in the last quarter.
  3. Total current asset value for each pharmacy storage vault.
  4. List of medications approaching their expiration window.
  5. Total sterilization maintenance cost per category of surgical tool.

Exercise 3:

Classify each of the corporate operational activities below by checking the correct infrastructure space: **Intranet**, **Extranet**, or **Internet**.

| N | Example of Activity | Intranet | Extranet | Internet |
|---|---|---|---|---|
| 1 | An accountant updating an internal branch-specific expense ledger. | | | |
| 2 | Giving trusted wholesale dealerships a portal to check available vehicle stock. | | | |
| 3 | A prospective client submitting an inquiry form via the public website. | | | |
| 4 | Uploading a team performance review template to the HR shared drive. | | | |
| 5 | an externalpartner log in and update delivery status timestamps directly in the company's fulfillment system. | | | |

Exercise 4:

Match each practical engineering task description below to its corresponding phase of the **Systems Development Life Cycle (SDLC)**:

| Task Description | SDLC Phase |
|---|---|
| 1. Running isolated stress tests on newly written database sub-routines. | |
| 2. Meeting with bank compliance officers to document regulatory security boundaries. | |
| 3. Finalizing structural wireframes, data dictionaries, and architectural maps. | |
| 4. Migrating historical user accounts and hosting technical training workshops for staff. | |
| 5. Writing modular, well-documented backend microservices in Python or C. | |

Exercise 5:

Complete the table below by filling in the correct specialized methodology or software management term for each description:

| Description | Term / Methodology |
| --- | --- |
| 1. An iterative approach characterized by deploying small, frequent functional updates within strictly bound, predictable timeboxes. | |
| 2. A structural shift or strategy course-correction executed by software product owners after analyzing real-world metrics from a lean release. | |
| 3. A traditional predictive development framework that is highly structured and best optimized for highly stable, deterministic enterprise systems. | |
| 4. A delivery approach where the absolute end-state system specifications cannot be mapped out or finalized during the initial project phases. | |
| 5. A fast-tracked development methodology relying heavily on interactive visual prototyping cycles directly involving target end-users. | |

Exercise 6:

Match the software engineering concept on the left with its defining characteristic on the right:

| Term | Definition / Key Feature |
| --- | --- |
| **1. Integrated Development Environment (IDE)** | **A.** Specialized suites engineered to automate code generation, model database designs visually, and streamline system analysis without low-level manual script-writing. |
| **2. Integration Testing** | **B.** A foundational, functional application build deployed rapidly to real-world target environments to collect maximum validated feedback with minimal effort. |
| **3. CASE Tools** | **C.** A unified engineering workspace combining text manipulation tools, semantic syntax validation, compilers, and internal execution tracers. |
| **4. Minimum Viable Product (MVP)** | **D.** A structured process of testing multiple combined software modules together to ensure they interact correctly and pass data seamlessly. |